

Modül 22

Security

XSS, CSRF, CSP, Sanitization

Modül:	22 / 30
Ders Sayısı:	7 ders
Seviye:	İleri
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 22'ye Hoş Geldin

Güvenlik — production app'in olmazsa olmazı. Bu modülde en yaygın web güvenlik açıklarını ve Angular'ın bunlara karşı sağladığı korumaları öğreneceksin.

Öğrenecekler:

- ✓ XSS (Cross-Site Scripting) — Angular'ın built-in koruması
- ✓ DomSanitizer — Trust types
- ✓ CSRF (Cross-Site Request Forgery) koruması
- ✓ Content Security Policy (CSP)
- ✓ Authentication best practices
- ✓ JWT handling
- ✓ Sensitive data ve secret management

XSS (Cross-Site Scripting)

En yaygın web güvenlik açığı. Angular nasıl korur?

⚠ XSS Nedir?

XSS: Saldırgan, başkasının browser'ında çalışacak kötü amaçlı script enjekte eder. Cookie çalabilir, kullanıcı adına işlem yapabilir, sayfayı manipüle edebilir.

📦 Angular'ın Otomatik Koruması

```
@Component({
  template: `<p>{{ userInput() }}</p>`
})
export class MyComponent {
  userInput = signal('<script>alert("XSS!")</script>');
  // Render: &lt;script&gt;alert("XSS!")&lt;/script&gt;
  // Script ÇALIŞMAZ – Angular otomatik escape eder
}
```

📦 Safe Contexts

Context	Otomatik Escape	Örnek
Interpolation {{ }}	Evet	{{ userInput }}
Property binding []	Evet	[innerText]="x"
innerHTML	Kısmen	[innerHTML]="..."
style/script srcs	Hayır	Manual sanitize

⚠ innerHTML Tehlikesi

```
// ☐ Tehlikeli
@Component({
  template: `<div [innerHTML]="userInput()"></div>`
})
export class Bad {
  userInput = signal('');
  // Angular bunu render ETMEZ – sanitize eder
  // Ama bypassSecurityTrustHtml ile bypass edilebilir
}

// ✓ Güvenli alternatif
@Component({
  template: `<div>{{ userInput() }}</div>` // Düz text
})
```

DomSanitizer — Trust Types

Trusted content ile çalışmak.

□ SecurityContext'ler

```
import { DomSanitizer, SecurityContext } from '@angular/platform-browser';

@Component({...})
export class MyComponent {
  private sanitizer = inject(DomSanitizer);

  // Manuel sanitize
  cleanHtml(dirty: string): string {
    return this.sanitizer.sanitize(SecurityContext.HTML, dirty) || '';
  }

  // SecurityContext'ler
  // SecurityContext.HTML      - innerHTML için
  // SecurityContext.STYLE     - [style] için
  // SecurityContext.URL       - href, src için
  // SecurityContext.SCRIPT    - script src için
  // SecurityContext.RESOURCE_URL - iframe, embed için
}
```

⚠ Bypass — Dikkat!

```
@Component({...})
export class MyComponent {
  private sanitizer = inject(DomSanitizer);

  // ⚠ Sadece TRUSTED kaynaktan gelen veri için
  trustedYouTubeUrl = this.sanitizer.bypassSecurityTrustResourceUrl(
    'https://www.youtube.com/embed/abc123'
  );

  // □ ASLA kullanıcıdan gelen veriyi bypass etme
  bypass(userInput: string) {
    return this.sanitizer.bypassSecurityTrustHtml(userInput); // □ Tehlikeli!
  }
}
```

□ Güvenli Markdown Render

```
// Markdown'ı sanitize edilmiş HTML'e çevir
import DOMPurify from 'dompurify';
import { marked } from 'marked';

@Pipe({ name: 'safeMarkdown', standalone: true })
export class SafeMarkdownPipe implements PipeTransform {
  private sanitizer = inject(DomSanitizer);

  transform(markdown: string): SafeHtml {
    const html = marked(markdown);
    const clean = DOMPurify.sanitize(html);
    return this.sanitizer.bypassSecurityTrustHtml(clean);
  }
}
```

CSRF (Cross-Site Request Forgery)

Başka siteden senin adına istek yapılmasını önlemek.

⚠️ CSRF Nedir?

Senario: banka.com'a login'sin. Saldırgan kötü-site.com'a girmeni sağlar. O site banka.com'a transfer isteği gönderir. Browser cookie'lerini eklediği için banka işlemi gerçekleştirir.

□ HttpClient'in Otomatik CSRF

```
// Angular HttpClient OTOMATİK olarak XSRF token ekler
// Backend'in cookie'de XSRF-TOKEN göndermesi yeterli

import { provideHttpClient, withXsrfConfiguration } from '@angular/common/http';

provideHttpClient(
  withXsrfConfiguration({
    cookieName: 'XSRF-TOKEN',      // Default
    headerName: 'X-XSRF-TOKEN',   // Default
  })
)

// Backend (Node.js Express örneği)
app.use(csurf({ cookie: true }));
app.use((req, res, next) => {
  res.cookie('XSRF-TOKEN', req.csrfToken());
  next();
});
```

□ SameSite Cookies

```
// Backend - cookie ayarları
res.cookie('session', token, {
  httpOnly: true,    // JS erişimi yok
  secure: true,      // Sadece HTTPS
  sameSite: 'strict' // Diğer site'lerden gelen isteklerle gönderilmez
});

// SameSite seçenekleri:
// 'strict' - Sadece aynı site
// 'lax'    - GET için ok, POST için strict
// 'none'   - Her zaman (secure: true gerek)
```

Content Security Policy

Browser'a "hangi kaynak güvenli" diyen header.

□ CSP Header

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'unsafe-inline' https://www.google-analytics.com;
  style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
  font-src 'self' https://fonts.gstatic.com;
  img-src 'self' data: https:;
  connect-src 'self' https://api.example.com;
  frame-ancestors 'none';
```

⚠ Angular'da CSP

```
// Angular default'da inline script üretebilir
// Strict CSP için nonce kullan:

// app.config.ts
import { provideExperimentalCheckNoChangesForDebug } from '@angular/core';

// index.html
<head>
  <meta http-equiv="Content-Security-Policy"
        content="script-src 'self' 'nonce-abc123'">
</head>

// Server-side: her request için unique nonce üret
// Angular: nonceAttribute'a ekle (production build ile)
```

Authentication Best Practices

Login, token, refresh, logout.

□ Token-Based Auth Flow

```

@Injected({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);
  private router = inject(Router);

  private _token = signal<string | null>(this.loadToken());
  private _user = signal<User | null>(null);

  readonly user = this._user.asReadonly();
  readonly isLoggedIn = computed(() => this._token() !== null);

  async login(email: string, password: string) {
    const { token, user } = await firstValueFrom(
      this.http.post<LoginResponse>('/api/auth/login', { email, password })
    );

    this.saveToken(token);
    this._token.set(token);
    this._user.set(user);

    this.router.navigate(['/dashboard']);
  }

  logout() {
    this.clearToken();
    this._token.set(null);
    this._user.set(null);
    this.router.navigate(['/login']);
  }

  getToken() { return this._token(); }

  private saveToken(token: string) {
    // ⚠ localStorage tehlikeli (XSS), httpOnly cookie tercih
    // Ama yine de çok yaygın
    localStorage.setItem('token', token);
  }

  private loadToken() {
    return localStorage.getItem('token');
  }

  private clearToken() {
    localStorage.removeItem('token');
  }
}

```

❑ Storage: localStorage vs httpOnly Cookie

Konu	localStorage	httpOnly Cookie
XSS riski	YÜKSEK	Yok (JS erişimi yok)

CSRF riski	Yok	Var (CSRF token gerek)
Send to API	Manuel	Otomatik
Mobile app	Kolay	Zor
Önerilen	☐ Sadece kısa süreli	✓ httpOnly + Secure + SameSite

JWT Handling

JWT decode, expire kontrol, refresh.

JWT Decode

```
// jwt-decode kütüphanesi
npm install jwt-decode

import { jwtDecode } from 'jwt-decode';

interface TokenPayload {
  sub: string;    // user id
  email: string;
  role: string;
  exp: number;    // expiration timestamp
  iat: number;    // issued at
}

const token = 'eyJhbGc...';
const decoded = jwtDecode<TokenPayload>(token);
console.log(decoded.email, decoded.role);
console.log(new Date(decoded.exp * 1000)); // Expire date
```

Expire Check

```
@Injectable({ providedIn: 'root' })
export class TokenService {
  isExpired(token: string): boolean {
    try {
      const decoded = jwtDecode<{ exp: number }>(token);
      return Date.now() >= decoded.exp * 1000;
    } catch {
      return true;
    }
  }

  willExpireSoon(token: string, minutes: number = 5): boolean {
    try {
      const decoded = jwtDecode<{ exp: number }>(token);
      const expireTime = decoded.exp * 1000;
      return Date.now() >= expireTime - (minutes * 60 * 1000);
    } catch {
      return true;
    }
  }
}
```

□ Token Refresh Pattern

```

@Injectables({...})
export class AuthService {
  private refreshing = false;
  private refreshPromise: Promise<string> | null = null;

  async getValidToken(): Promise<string> {
    const token = this.getToken();

    if (!token) throw new Error('No token');

    if (!this.tokenService.willExpireSoon(token)) {
      return token;
    }

    // Concurrent refresh requests'i tek istek olarak yap
    if (this.refreshPromise) {
      return this.refreshPromise;
    }

    this.refreshPromise = this.refreshToken();
    try {
      const newToken = await this.refreshPromise;
      return newToken;
    } finally {
      this.refreshPromise = null;
    }
  }

  private async refreshToken(): Promise<string> {
    const { token } = await firstValueFrom(
      this.http.post<{ token: string }>('/api/auth/refresh', {})
    );
    this.saveToken(token);
    this._token.set(token);
    return token;
  }
}

```

Sensitive Data & Secret Management

API key, password, ve diğer sensitive bilgiler.

❏ Frontend'de Secret Yok

⚠️ Önemli Kural

Frontend'e konan her şey kullanıcı tarafından görülebilir. API key'ler, secret'lar asla frontend'de bulunamaz. Tüm secret'lar backend'de tutulmalı.

❏ Environment Variables

```
// environments/environment.ts (dev)
export const environment = {
  production: false,
  apiUrl: 'http://localhost:3000/api',
  publicApiKey: 'pk_test_...', // Sadece public key'ler
};

// environments/environment.prod.ts (production)
export const environment = {
  production: true,
  apiUrl: 'https://api.example.com',
  publicApiKey: 'pk_live_...',
};

// ❏ ASLA backend secret'larını koyma
const SECRET_KEY = 'sk_live_XXXXXX'; // ❏ Frontend'de görülür!
```

❏ Best Practices

- ✓ Public key'ler: Frontend'de OK (Stripe pk, Mapbox public token)
- ✓ Secret key'ler: Sadece backend
- ✓ OAuth: Backend-for-Frontend pattern
- ✓ API çağrıları: Hassas data backend üzerinden
- ✓ Auth header: Token expire kısa, refresh ile yenile

❏ Genel Security Checklist

- ✓ HTTPS zorunlu (HTTP'ye redirect)
- ✓ CSP header set edilmiş
- ✓ XSS koruması aktif (Angular default)

- ✓ CSRF token (HttpClient default)
- ✓ httpOnly + Secure + SameSite cookie
- ✓ JWT expire kısa (15 min) + refresh token
- ✓ Password'ler hashed (backend, bcrypt/argon2)
- ✓ Rate limiting (backend)
- ✓ Input validation (frontend + backend)
- ✓ Error messages sensitive bilgi vermesin
- ✓ Dependencies güncel (npm audit)

☐ Modül 22 Özeti

Güvenlik konusunda artık donanımlıydın. Angular'ın built-in koruması, manuel security pattern'ler, JWT handling — hepsi cebinde.

☐ Neler Öğrendin?

- ✓ XSS — Angular'ın otomatik koruması
- ✓ DomSanitizer — Trust types ve bypass
- ✓ CSRF — HttpClient otomatik koruma
- ✓ CSP — Content Security Policy
- ✓ Authentication best practices
- ✓ JWT decode, expire check, refresh
- ✓ Secret management — backend'de tut

☐ Sıradaki Modül

Modül 23 — Build & Deployment. Production build, Docker, CI/CD pipelines.